

GramDrishti

Backend guide: data, models, advisories, API

Build the intelligence: from block forecast to calibrated Panchayat forecast to reviewed advisory. Written for one person, from an empty folder.

You are the backend person. You own data loading, features, the downscaling models, uncertainty, the agro-variables, the advisory rules, the database, the verification job and the API.

Read in this order: Project snapshot, then Sections 1 to 14, then Appendix A (contract) and Appendix B (integration).

What you can trust: the mock dataset from `gramdrishti_mock_data.zip`. It is synthetic. Anything that looks like accuracy on it only shows the code works.

Contents: 1 Your deliverables. 2 Setup. 3 Data layer. 4 Features. 5 Baselines and bias correction. 6 Downscaling model. 7 Uncertainty and consistency. 8 Agro-variables. 9 Advisory engine. 10 Database and review workflow. 11 Verification and impact. 12 API layer. 13 Extras. 14 Testing, pitfalls, timeline, switching to real data.

Project snapshot (both people read this)

GramDrishti turns a coarse block-level weather forecast into Panchayat-level forecasts with uncertainty, then into crop-stage advisories that an officer reviews before farmers receive them in Hindi, Punjabi or English. We prove the value by comparing against the plain block forecast.

Who owns what

	Backend person	Frontend person
Owens	Data, features, models, uncertainty, advisory rules, database, verification, API	Officer console, farmer app, map, charts, bulletin print page, design system, languages
Delivers	A running API at /api/v1 plus example JSON files that match the contract	A running web app that works on example JSON first, then on the real API
Never touches	React code, CSS, map styling	Model code, rules, database
Owens together	The API contract (Appendix A), integration (Appendix B), demo script	

Repository layout (one shared Git repo)

```
gramdrishti/  
  contract/      <- SHARED. openapi.json, examples/*.json, CHANGELOG.md  
  data/          <- mock data from the zip (panchayats_static.csv, etc.)  
  backend/       <- backend person only  
  frontend/      <- frontend person only  
  docs/          <- this PDF set, model card, demo script  
  docker-compose.yml <- added at integration time  
  README.md
```

Ground rules

- **Contract first.** The JSON shapes in Appendix A are frozen on Day 2. After that, any change goes through a pull request that both people approve and that adds a line to contract/CHANGELOG.md.
- **Honesty on screen.** All current data is synthetic. Every API response carries data_mode (mock or real). When it is mock, the UI shows a visible ribbon: "Synthetic demo data".
- **No invented accuracy.** Metrics shown in the UI come only from the verification job. Fixtures may hold obviously fake numbers, always with data_mode: mock.
- **Thresholds are placeholders** until a KVK or agriculture-university expert reviews them. Advisory cards carry thresholds_status, and the UI shows a small "pending expert review" label when it is placeholder.
- **Conventions.** JSON keys in snake_case. Dates as ISO YYYY-MM-DD. Coordinates as [longitude, latitude] in GeoJSON. Probabilities 0 to 1. Units fixed per variable (rain mm, temperature C, humidity %, wind km/h).
- **Working rhythm.** 15-minute sync every day. One integration branch. Merge to main only when both sides run.

Time plan assumes about 4 weeks of part-time work. If you have fewer weeks, cut the items marked "extra" first, never the core loop: forecast -> uncertainty -> advisory -> review -> verification.

1. Your deliverables and definition of done

Deliverable	Definition of done
Data layer	One function per file loads mock or real data with the same column names; QC flags bad values
Feature builder	Produces a training table and an inference table with no use of future information
Baselines B0, B1, B2	Three simple predictors that the model must beat
Downscaling model	Panchayat p10, p50, p90 and event probabilities for rain, Tmax, Tmin, RH, wind for lead days 1 to 5
Calibration and consistency	Conformal-adjusted intervals; Panchayat values average back to the block forecast exactly
Agro-variables	ET0, soil water, GDD, heat/cold stress, waterlogging risk, livestock THI per Panchayat and day
Advisory engine	YAML rules produce cards in three languages with reason, evidence, confidence and fallback
Review workflow	SQLite tables; approve/edit/reject writes an audit log
Verification + impact	A single command writes <code>verification.json</code> and <code>impact.json</code> from held-out data
API	Every endpoint in Appendix A returns valid JSON matching the contract
Snapshots	A command precomputes the demo issue dates so the API answers instantly

2. Setup from scratch

2.1 Install

- Python 3.11 or 3.12, Git, VS Code (or any editor). Windows users: use WSL2 or plain PowerShell, both work.
- Unzip `gramdrishti_mock_data.zip` into `gramdrishti/data/` so that `gramdrishti/data/panchayats_static.csv` exists.

2.2 Create the project

```
mkdir gramdrishti && cd gramdrishti && git init
mkdir -p contract/examples data backend docs
cd backend
python -m venv .venv
source .venv/bin/activate                # Windows: .venv\Scripts\activate
pip install fastapi "uvicorn[standard]" pandas numpy pyarrow lightgbm scikit-learn shap joblib
pip install pydantic pyyaml simpleeval gtts pytest httpx
pip freeze > requirements.txt            # pin versions once everything works
mkdir -p gramdrishti/{data,features,models,agro,advisory,store,verify,pipeline,api} tests artifacts
touch gramdrishti/__init__.py gramdrishti/{data,features,models,agro,advisory,store,verify,pipeline,api}
/__/init__.py
```

2.3 Folder map (what goes where)

Folder	Contents
gramdrishti/data/	loaders.py, qc.py, config.py
gramdrishti/features/	static.py, dynamic.py, build.py
gramdrishti/models/	baselines.py, bias.py, downscale.py, conformal.py, reconcile.py, explain.py
gramdrishti/agro/	derived.py (ET0, soil water, GDD, THI, waterlogging)
gramdrishti/advisory/	rules.yaml, templates.yaml, signals.py, engine.py, spray.py
gramdrishti/store/	db.py (SQLite), audit log, feedback
gramdrishti/verify/	metrics.py, run_validation.py, impact.py
gramdrishti/pipeline/	train.py, run_daily.py (snapshot builder)
gramdrishti/api/	main.py, schemas.py, routes/*.py
artifacts/	Trained models, snapshots, verification.json (git-ignored except small examples)

Golden rule for the whole backend: **anything that runs at request time only reads precomputed files or the database.** Training and forecasting happen offline in `train.py` and `run_daily.py`.

3. Data layer

Goal: the rest of the code never knows whether data is mock or real. It only sees DataFrames with fixed column names.

```
# gramdrishti/data/config.py
import os
from pathlib import Path
DATA_MODE = os.getenv("DATA_MODE", "mock") # "mock" or "real"
ROOT = Path(__file__).resolve().parents[3]
DATA = ROOT / "data"
ART = ROOT / "backend" / "artifacts"
LEADS = [1, 2, 3, 4, 5]
VARS = ["rain", "tmax", "tmin", "rh", "wind"]
COLS = {"rain": "rain_mm", "tmax": "tmax_c", "tmin": "tmin_c", "rh": "rh_mean_pct", "wind": "wind_kmh"}

# gramdrishti/data/loaders.py
import pandas as pd
from .config import DATA
def _csv(name, **kw): return pd.read_csv(DATA / name, **kw)
def load_static(): return _csv("panchayats_static.csv")
def load_stations(): return _csv("stations.csv")
def load_obs(): return _csv("station_observations_mock.csv", parse_dates=["date"])
def load_fc(): return _csv("block_forecast_mock_nwp.csv", parse_dates=["valid_date",
"issue_date"])
def load_sat(): return _csv("satellite_weekly_mock.csv", parse_dates=["week_start"])
def load_crops(): return _csv("panchayat_crops_mock.csv", parse_dates=["sowing_date"])
def load_calendar(): return _csv("crop_calendar_PLACEHOLDER.csv")
def load_truth(): # MOCK ONLY. Panchayat truth does not exist in real life.
return _csv("synthetic_oracle/panchayat_daily_SYNTHETIC_TRUTH.csv", parse_dates=["date"])
def load_block_truth():
return _csv("synthetic_oracle/block_daily_SYNTHETIC_TRUTH.csv", parse_dates=["date"])
```

3.1 Quality control (qc.py)

Add a flag column, never silently delete. Checks that matter for the plains and for our advisories:

Check	Rule
Range	rain 0 to 400 mm; tmax -5 to 52; tmin -10 to 35; rh 0 to 100; wind 0 to 120
Consistency	tmin must be below tmax; flag days where tmax - tmin < 1 or > 25
Spike	Temperature jump above 12 C from the previous day at a station

Check	Rule
Stuck sensor	Same non-zero value for 5 or more days
Missing	Count per station per month; expose through /data-quality

Mock versus real. `load_truth()` exists only in mock mode. In real mode it raises a clear error. Anywhere you use it, write a comment `# MOCK ONLY` so nobody trains the final model on it by accident.

4. Features

Build one function `make_table(kind)` returning rows of (`valid_date`, `lead_day`, `panchayat_id`) with features and, for training, targets. `kind` is `train` or `infer`. The same feature code runs in both, which prevents training/serving mismatch.

Group	Features	Source
Static local	elevation, <code>tpi_z</code> , slope, irrigated_frac, canal_dist_km, cropland/urban/water/tree fractions, clay/sand/silt, whc, drainage class (encoded)	<code>panchayats_static.csv</code>
Position	lat, lon, block centre offset (Panchayat minus block centroid)	static + blocks
Calendar	sin/cos of day-of-year, month, <code>lead_day</code>	computed
Forecast context	bias-corrected block forecast of all 5 variables, spread between the two NWP sources (a crude uncertainty), day-to-day change	forecast files
Recent surface state	latest NDVI and land surface temperature (last available week before the issue date), NDVI trend	<code>satellite_weekly_mock.csv</code>
Recent weather	block rainfall in previous 3 and 7 days (from observations available at issue time)	obs / gridded

Rules that prevent leakage

- A feature for issue date `D` may use data available **before or on** `D` only. NDVI: use the latest week that ended before `D`.
- Never use `synthetic_oracle` values as features. They are the answer key.
- Use the issued forecast at lead `L` as the block value at inference. Do not use block truth as a feature, because it does not exist in real time.

```
# gramdrishti/features/dynamic.py (latest satellite value before each issue date)
def latest_before(sat, issue_dates, pid_col="panchayat_id"):
    sat = sat.sort_values("week_start")
    out = []
    for D in issue_dates:
        s = sat[sat.week_start + pd.Timedelta(days=7) <= D]
        last = s.dropna(subset=["ndvi"]).groupby(pid_col).tail(1)
        out.append(last.assign(issue_date=D)[[pid_col, "issue_date", "ndvi", "lst_day_c"]])
    return pd.concat(out)
```

5. Baselines and bias correction

Baselines come first. If you cannot beat them, the model is not useful, and it is better to find that out now.

Baseline	Definition	Why it matters
B0 raw block	Average of the two NWP sources at the block, copied unchanged to every Panchayat in that block	This is what is done today. The jury's question is: do we beat this?
B1 corrected block	B0 after bias correction (below), copied to every Panchayat	Shows the gain from spatial downscaling alone, not from fixing bias
B2 simple physical	B1 plus a lapse-rate term (-6.5 C per km of elevation) and inverse-distance weighted station anomalies	Shows the gain from machine learning over simple physics

5.1 Bias correction (models/bias.py)

- **Temperature and humidity:** additive mean bias per source, lead day and season group, fitted on block-level forecast versus block truth over the training period only.
- **Wind:** multiplicative ratio per source and lead.
- **Rain:** empirical quantile mapping on wet days, after matching wet-day frequency. Fit separately for monsoon (Jun to Sep) and other months.
- **Combine sources:** weighted mean with weights proportional to $1 / \text{MSE}$ on the training period. Keep the spread between sources as a feature.

```
import numpy as np
def fit_qm(fc, obs, n=99):
    qs = np.linspace(0.01, 0.99, n)
    return np.quantile(fc, qs), np.quantile(obs, qs)
def apply_qm(x, fq, oq):
    return np.interp(x, fq, oq) # flat extrapolation outside range

def fit_rain_qm(fc, obs, wet_thr=1.0):
    wet_freq = (obs >= wet_thr).mean()
    cut = np.quantile(fc, 1 - wet_freq) # forecast amount that gives the same wet
    frequency
    m = fc >= cut
    return {"cut": cut, "qm": fit_qm(fc[m], obs[obs >= wet_thr])}
def apply_rain_qm(x, p):
    y = np.zeros_like(x, dtype=float); m = x >= p["cut"]
    y[m] = apply_qm(x[m], p["qm"]); return y
```

Verify: after correction, the mean bias of the block forecast on the calibration period is near zero, and wet-day frequency matches. Print both, do not assume.

6. Downscaling model (models/downscale.py)

6.1 What we predict

The corrected block forecast already carries the big weather pattern. The model learns **how a specific Panchayat differs from it**.

Variable	Target	Models trained (LightGBM)
tmax, tmin	anomaly = Panchayat truth - corrected block forecast	mean (L2) + quantile 0.1, 0.5, 0.9
rh	anomaly for dew point, then RH is derived from downscaled T and dew point (keeps them physically consistent)	same as temperature
wind	log ratio = $\log(\text{Panchayat} / \text{block forecast})$	mean + quantiles
rain amount	Panchayat rain in mm (zero-heavy)	mean + quantiles 0.1, 0.5, 0.9
rain events	binary: rain $\geq$ 1, 2.5, 10, 35 mm	one classifier per threshold, then isotonic calibration

Training uses **mock truth at all Panchayats**. That is the proxy setup. In real mode the target comes only from the station Panchayats (see Section 14).

6.2 Time split (write it down once, reuse everywhere)

```
TRAIN = ("2023-01-01", "2024-04-30")      # fit all models
CALIB = ("2024-05-01", "2024-07-15")      # conformal + isotonic calibration only
TEST  = ("2024-07-16", "2024-12-31")      # never touched until the final verification run
# Purge: drop rows whose lead-day window overlaps the next split (5 days) to avoid leakage.
```

Also run leave-one-block-out on the training period (6 folds: hide all Panchayats of one block, train on the rest). This tests generalisation to unseen locations and avoids spatial leakage between neighbouring Panchayats.

6.3 Training code pattern

```
import lightgbm as lgb
BASE = dict(n_estimators=400, learning_rate=0.05, num_leaves=31, min_child_samples=50,
            subsample=0.8, subsample_freq=1, colsample_bytree=0.8, n_jobs=-1, verbose=-1)

def fit_var(X, y, alphas=(0.1, 0.5, 0.9)):
    models = {"mean": lgb.LGBMRegressor(objective="regression", **BASE).fit(X, y)}
    for a in alphas:
        models[a] = lgb.LGBMRegressor(objective="quantile", alpha=a, **BASE).fit(X, y)
    return models

def fit_event(X, y_bool):
    return lgb.LGBMClassifier(**BASE).fit(X, y_bool)
```

- Save with `joblib.dump(models, ART / f'model_{var}.joblib')` and a JSON file listing feature names in order.
- Categorical `drainage_class` and `soil_texture`: encode to integers with a saved mapping.
- Start with the defaults above. Tune only `num_leaves`, `min_child_samples` and `n_estimators` on the calibration period if time allows.
- Sanity checks after training: predicted Tmax anomaly should be lower for high `irrigated_frac`; rain quantiles must not be negative.

7. Uncertainty and block consistency

7.1 Conformalised quantile regression (models/conformal.py)

Raw quantile models are often too narrow or too wide. We calibrate them on the calibration period so the stated interval is honest. For an 80% interval built from p10 and p90:

```
def cqr_offset(lo, hi, y, level=0.8):
    score = np.maximum(lo - y, y - hi)                # how far y falls outside [lo, hi]
    n = len(y)
    k = min(1.0, np.ceil((n + 1) * level) / n)
    return np.quantile(score, k)                      # final interval: [lo - q, hi + q]
```

- Compute one offset per **variable and lead-day group** (day 1-2, day 3-5) and per season group if data allows.
- After adding the offset, force order: `p10 <= p50 <= p90`, `rain >= 0`, `RH` within 0 to 100.
- Report empirical coverage on the test period next to the nominal 80%. If far off, say so and investigate. Do not hide it.

7.2 Reconcile with the block forecast (models/reconcile.py)

The average of Panchayat values inside a block must equal the corrected block forecast. Use the **mean model output** to compute the correction, then apply it to the mean and all quantiles.

```
def reconcile(df, target_col, mode):
    # df rows: one (valid_date, lead_day, panchayat) each; columns: block_id, mean, p10, p50, p90,
    # target
    g = df.groupby(["valid_date", "lead_day", "block_id"])["mean"].transform("mean")
    cols = ["mean", "p10", "p50", "p90"]
    if mode == "add":                                # tmax, tmin, dew point
        shift = df[target_col] - g
        for c in cols: df[c] = df[c] + shift
```

```

else:
    ratio = np.where(g > 1e-6, df[target_col] / g, 1.0)
    for c in cols: df[c] = df[c] * ratio
return df

```

Test it: after reconcile, the block mean of mean minus the target must be below 1e-6 for every block-day. This test is one of your strongest demo points: we redistribute the official forecast, we never contradict it.

With equal Panchayat weights in the prototype, the block mean is the plain average. In production use polygon-area weights.

8. Agro-variables (agro/derived.py)

Quantity	Formula (daily)	Notes
ET0	$0.0023 * (T_{\text{mean}} + 17.8) * \sqrt{T_{\text{max}} - T_{\text{min}}} * Ra$	Hargreaves. Ra is extraterrestrial radiation by latitude and day of year (FAO-56). Replace with Penman-Monteith when wind, RH and radiation are trusted.
Growing degree days	$\max((T_{\text{max}} + T_{\text{min}})/2 - T_{\text{base}}, 0)$	Tbase per crop from the calendar table (placeholder until reviewed)
Soil water	$SM(t+1) = \text{clip}(SM(t) + \text{rain_eff} - kc * ET0 * \text{stress}, 0, \text{capacity})$	capacity = whc_mm_per_m * 0.6 m. rain_eff removes runoff above 20 mm. Start from the latest known state.
Depletion fraction	$1 - SM / \text{capacity}$	Drives irrigation advice
Waterlogging risk	$P(\text{rain} \geq 35 \text{ mm})$ combined with drainage poor or tpi_z < -0.7, and SM above 90% of capacity	Levels low / moderate / high / severe from the score
Livestock THI	$0.8 * T + (RH/100) * (T - 14.4) + 46.4$	Use Tmax and afternoon RH proxy. Thresholds by animal type need expert review.
Frost/fog risk	$P(T_{\text{min}} \leq \text{crop frost threshold})$ and tpi_z low, or Dec-Jan fog proxy	Uses p10 of Tmin
Dry-spell counter	Consecutive days with $P(\text{rain} \geq 1 \text{ mm}) < 0.2$	Needs 5-day forecast

Run soil water forward for five days using **p50 rain** for the central path, and also with p10 and p90 rain to get a range. Store results in the snapshot.

Mock caveat: the initial soil moisture comes from the mock oracle. In real mode take it from ERA5-Land or SMAP soil moisture, or from a bucket model run on observed rain.

9. Advisory engine

The engine is **rules + signals + templates**, not a language model. This makes every advisory traceable and lets an expert edit thresholds without touching Python. An LLM can later polish wording in review, but it never decides.

9.1 Pipeline

1. For each Panchayat, take its crops from panchayat_crops_mock.csv and estimate the stage: `das = valid_date - sowing_date`, then look up the stage in crop_calendar_PLACEHOLDER.csv.
2. Build a **signals** dictionary from the forecast and agro-variables (list below).
3. Evaluate every rule for that crop and stage with `simpleeval` (safe expression evaluator).
4. For each rule that fires, fill the template in en, hi, pa and attach evidence and confidence.
5. Save as draft advisories. Officers review before farmers see them.

9.2 Signals

Signal	Meaning
p_rain_2_5_d1, p_rain_2_5_2d	Probability of ≥ 2.5 mm on day 1, and on either of the next two days
p_rain_10_3d, p_rain_35_d1	Probability of ≥ 10 mm in next 3 days; ≥ 35 mm tomorrow
tmax_p90_3d, tmin_p10_3d	Highest p90 Tmax and lowest p10 Tmin over the next 3 days
wind_p90_d1	p90 wind tomorrow
depletion	Soil water depletion fraction at start of window
dry_days	Length of forecast dry spell in days
rh_mean_3d, warm_wet_index	Humidity and warmth used for disease risk (rules placeholder)
thi_p90	Livestock heat index p90
stage, das, crop	From calendar and sowing date
drain_poor, low_lying, texture	From static features

9.3 Rule file (advisory/rules.yaml)

```
- id: wheat_terminal_heat
  crop: wheat
  when: "stage == 'flowering_grain_fill' and tmax_p90_3d >= 34"          # threshold = PLACEHOLDER
  category: heat_stress
  priority: high
  template: wheat_heat
  thresholds_status: placeholder
  source: ""                                                            # expert fills reference

- id: spray_window_bad
  crop: any
  when: "p_rain_2_5_2d >= 0.35 or wind_p90_d1 >= 15"
  category: spray
  priority: moderate
  template: spray_hold
  thresholds_status: placeholder

- id: irrigate_now
  crop: any
  when: "depletion >= 0.55 and p_rain_10_3d < 0.25 and stage in irrigation_sensitive_stages"
  category: irrigation
  priority: high
  template: irrigate
```

9.4 Starter rule set (all thresholds are placeholders)

Rule	Fires when	Action in one line
Sowing window	Soil moisture adequate, low P(heavy rain 2 d), sowing season open	Sow now / delay if dry spell or heavy rain likely
Irrigation	Depletion high and low chance of useful rain	Irrigate; hold irrigation if rain likely (saves water and pump energy)
Spray hold	P(rain >= 2.5 mm, next 2 d) high or wind p90 high	Do not spray or top-dress; wash-off and drift risk
Fertilizer timing	Rain likely within 24 h, or waterlogging risk	Postpone urea; apply when a dry window is forecast
Heat stress	Tmax p90 above crop-stage threshold	Light irrigation, avoid stress operations
Frost / cold	Tmin p10 below threshold, low-lying Panchayat	Light irrigation, cover nurseries
Waterlogging	Heavy rain probable and poor drainage or low ground	Clear drains, hold fertilizer, protect seedlings
Dry spell	Dry days >= 5 and high ET0 at a sensitive stage	Prioritise critical-stage irrigation, mulch
Harvest	Crop mature and rain risk window ahead	Harvest early or delay; protect produce
Pest / disease	Warm humid run for several days at susceptible stage	Scout fields; expert-written rule
Livestock heat	THI p90 above threshold	Shade, water, avoid midday grazing

9.5 Confidence and priority

```
def confidence(prob_margin, lead_day, interval_width_ratio):
    # prob_margin: how far the key probability is beyond the trigger threshold (0..1)
    score = prob_margin - 0.08 * (lead_day - 1) - 0.3 * interval_width_ratio
    return "high" if score > 0.25 else "medium" if score > 0.05 else "low"
# priority: severity of loss for that stage (from calendar table) x confidence; keep it transparent.
```

9.6 Templates and languages

Keep every sentence in `templates.yaml` with placeholders and three languages. Write the English first, then Hindi and Punjabi, then get a native speaker or agriculture staff member to review. Do not machine-translate agronomy terms without checking. Example structure:

```
irrigate:
  action: {en: "Irrigate on {day}. Skip if you irrigated in the last {gap_days} days.", hi: "...",
    pa: "..."}
  reason: {en: "Soil is drying ({depletion_pct}% used) and only {p_rain_pct}% chance of useful
    rain.", hi: "...", pa: "..."}
  fallback: {en: "If more than 5 mm of rain falls overnight, skip irrigation.", hi: "...", pa: "..."}
```

9.7 Spray planner (day level)

The mock data is daily, so the prototype outputs a **day suitability** per Panchayat: Good, Caution, Avoid, based on P(rain >= 2.5 mm) today and tomorrow and wind p90. State clearly that hour-level windows need hourly NWP (GFS provides 3-hourly fields) and are future work.

10. Database and review workflow (store/db.py)

```
CREATE TABLE advisories(id TEXT PRIMARY KEY, issue_date TEXT, panchayat_id TEXT, crop TEXT, stage TEXT,
category TEXT, priority TEXT, valid_from TEXT, valid_to TEXT, payload_json TEXT, status TEXT,
reviewed_by TEXT, reviewed_at TEXT);
CREATE TABLE audit_log(id INTEGER PRIMARY KEY AUTOINCREMENT, advisory_id TEXT, action TEXT, actor TEXT,
before_json TEXT, after_json TEXT, note TEXT, at TEXT);
CREATE TABLE feedback(id INTEGER PRIMARY KEY AUTOINCREMENT, panchayat_id TEXT, date TEXT,
reported_rain INTEGER, intensity TEXT, channel TEXT, at TEXT);
CREATE TABLE farmers(id TEXT PRIMARY KEY, name TEXT, panchayat_id TEXT, language TEXT, crops_json TEXT);
```

- **States:** draft -> approved | edited | rejected. Only approved and edited are returned to farmers.
- Every review call writes one row to audit_log with before and after JSON, the actor and the time.
- Seed 5 demo farmers in different Panchayats with different crops and languages (seed_demo.py). Name them clearly as demo profiles.
- Use plain sqlite3 or SQLAlchemy. SQLite is fine for the prototype; Postgres/PostGIS is the production statement.

11. Verification and impact (verify/)

This is your most important module. The jury asks one question: **is the Panchayat forecast better than the block forecast?** Everything here answers it honestly.

11.1 Protocol

Test	How	What it shows
Temporal holdout	Train and calibrate before the TEST window; evaluate only on TEST	Works on future days
Leave-one-block-out	Retrain 6 times, each hiding a whole block	Works at unseen locations (spatially honest)
Station check	Compare with noisy station values at the 24 station Panchayats	Works against measurement-like data
Stratified	By lead day, season, rain intensity, drainage class	Where it helps and where it does not
Decision level	Replay spray, irrigation and heat decisions using model vs block-level forecast	Impact on advice, not only on numbers

11.2 Metrics

Aspect	Metrics
Continuous (tmax, tmin, rh, wind)	MAE, RMSE, bias; skill = 1 - MAE_model / MAE_B0
Rain occurrence	POD, FAR, CSI (threshold 1, 2.5, 10, 35 mm); frequency bias
Rain amount	MAE on wet days, quantile loss
Probability quality	Brier score, Brier skill versus climatology, reliability points, PIT or coverage
Intervals	Empirical coverage of the 80% interval, average width
Integrity	Block-mean error (should be about zero)

```
def contingency(obs, fc):
    # boolean arrays
    h = (obs & fc).sum(); m = (obs & ~fc).sum(); f = (~obs & fc).sum()
    pod = h / (h + m) if h + m else np.nan
    far = f / (h + f) if h + f else np.nan
    csi = h / (h + m + f) if h + m + f else np.nan
    return pod, far, csi

def brier(p, o): return np.mean((p - o) ** 2)

def reliability(p, o, bins=10):
    b = np.clip((p * bins).astype(int), 0, bins - 1)
```

```

pts = []
for i in range(bins):
    m = b == i
    if m.any():
        pts.append({"forecast_prob": (i + .5) / bins,
                    "observed_freq": float(o[m].mean()), "n": int(m.sum())})
return pts

def block_bootstrap_ci(day_err_model, day_err_base, block=7, n_boot=1000, seed=0):
    # inputs: one mean absolute error per day (average over Panchayats first), ordered by date
    rng = np.random.default_rng(seed); n = len(day_err_model)
    starts = np.arange(0, n - block + 1)
    out = []
    for _ in range(n_boot):
        pick = rng.choice(starts, size=n // block)          # resample 7-day blocks with replacement
        idx = np.concatenate([np.arange(s, s + block) for s in pick])
        out.append(1 - day_err_model[idx].mean() / day_err_base[idx].mean())
    return np.percentile(out, [2.5, 97.5])                # 95% interval for skill vs baseline

```

Run `python -m gramdrishti.verify.run_validation` to write `artifacts/verification.json` in the shape of Appendix A. Include notes that say: synthetic truth, proxy validation, and list every variable where the model does **not** beat B0.

11.3 Impact replay (impact.py)

Pick decisions with a clear outcome and replay them over the TEST period for every Panchayat.

Decision	Model rule	Block-level rule	Outcome from truth
Spray tomorrow?	Advise spray if $P(\text{rain} \geq 2.5 \text{ mm}) < 0.3$	Advise spray if block forecast rain $< 2.5 \text{ mm}$	Actual Panchayat rain $\geq 2.5 \text{ mm}$ means washed off
Heat alert	Alert if $p90 \text{ Tmax} \geq \text{threshold}$	Alert if block $\text{Tmax} \geq \text{threshold}$	Actual Panchayat $\text{Tmax} \geq \text{threshold}$
Irrigate or wait	Wait if $P(\text{rain} \geq 5 \text{ mm}) \geq 0.5$	Wait if block rain $\geq 5 \text{ mm}$	Actual rain $\geq 5 \text{ mm}$

Count: correct advice, wasted wait (told to wait, stayed dry), washed off (told OK, rain came). Report **counts and rates only**, no money values, until an expert supplies costs.

12. API layer (api/)

12.1 Structure

```

# gramdrishti/api/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
app = FastAPI(title="GramDrishhti API", version="0.1.0")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173"],
                  allow_methods=["*"], allow_headers=["*"])
from .routes import meta, geo, forecast, advisories, verification, farmers # each exposes a router
for r in (meta, geo, forecast, advisories, verification, farmers):
    app.include_router(r.router, prefix="/api/v1")

# gramdrishti/api/schemas.py (Pydantic models mirror Appendix A exactly)
from pydantic import BaseModel
from typing import Optional
class Quantiles(BaseModel):
    p10: Optional[float]; p50: Optional[float]; p90: Optional[float]; block: Optional[float]

```

12.2 Rules for every route

- Read from `artifacts/snapshots/{issue_date}/*.parquet` and SQLite. **No model inference per request.**
- Return Pydantic models; convert NaN with `df.replace({np.nan: None})` before building responses.
- Raise `HTTPException(404, detail={'error': {'code': 'not_found', 'message': '...'}})` in the contract error shape (add an exception handler).
- Add `response_model` to every route so `openapi.json` is exact. Export it with `python -m gramdrishti.export_openapi`.
- Serve GeoJSON from files written once by a script (`panchayats_SYNTHETIC.geojson`). Simplify geometry if payload is large.

12.3 Snapshot builder (pipeline/run_daily.py)

```
python -m gramdrishti.pipeline.train # fits bias, models, conformal offsets
python -m gramdrishti.pipeline.run_daily --issue-date 2024-08-12
# 1 build inference table 2 bias-correct block forecast 3 predict + calibrate + reconcile
# 4 derive agro variables 5 compute risks and priority 6 generate draft advisories
# 7 write artifacts/snapshots/2024-08-12/{forecast,risk,priority}.parquet
python -m gramdrishti.pipeline.run_daily --all-demo-dates # loop over the chosen demo dates
```

Pick 6 to 8 demo issue dates that show different situations: a heavy monsoon day, a dry spell, a hot pre-monsoon day, a foggy cold morning in Dec-Jan, an ordinary day. `/meta.available_issue_dates` lists them. The frontend gets a time-travel demo for free, and `/observed` lets it show what really happened afterwards.

13. Extras (do these after the core loop works)

Extra	How	Effort
Explain (SHAP)	<code>shap.TreeExplainer(mean_model)</code> on one row; take top 3 by absolute value; map feature and sign to a plain sentence from a dictionary (for example irrigated_frac negative on tmax -> 'More irrigated fields around it')	Small
Forecast-change tracker	Keep the previous snapshot. For the same valid date compare p50 and event probabilities; flag when an advisory category changes	Small
Live observation nudge	If today's station value differs from the day-1 forecast, shift the next day's p50 by $0.3 * \text{residual} * \exp(-\text{lead}/2)$. Mark as heuristic	Small
Audio	<code>gTTS(text, lang='hi')</code> saved to <code>artifacts/audio/</code> . Check that pa is supported in your installed version; if not, frontend falls back to browser speech	Small
Data-quality endpoint	Count stations with data in the last 24 h, missing share, stale inputs	Small
Pest and disease rules	Rules for a few crops written with an expert's numbers, marked placeholder until reviewed	Medium
Livestock THI advisory	Uses derived THI, one template	Small

14. Testing, pitfalls, timeline, switching to real data

14.1 Tests to write (pytest)

- **Consistency:** block mean of reconciled forecasts equals the block target within $1e-6$.
- **No leakage:** max training date < min calibration date < min test date; feature dates never exceed issue date.
- **Ordering:** $p_{10} \leq p_{50} \leq p_{90}$; $\text{rain} \geq 0$; RH within 0 to 100; $T_{\min} < T_{\max}$.
- **Rules:** one small fixture per rule proves it fires and does not fire when it should not.
- **API:** every endpoint returns 200 for the demo date and validates against its Pydantic model; no NaN in JSON.
- **Determinism:** a fixed seed reproduces `verification.json`.

14.2 Pitfalls

Pitfall	What to do
Random train/test split	Never. Always split by time and by block
Testing on the calibration window	Keep three separate windows and never mix
Using block truth at inference	Only the issued forecast exists in real time
Intervals look great because they are huge	Report width next to coverage
Model does not beat B0 for a variable	Report it. Say why (for example flat terrain). Do not tune until it wins on the test set
Over-trusting mock results	Every verification screen and document states: synthetic proxy validation
Slow API	Precompute snapshots

14.3 Suggested timeline (about 20 working days)

Days	Tasks	You can show
1 - 2	Setup, loaders, QC, contract examples exported	<code>contract/examples/*.json</code> for frontend
3 - 5	Feature builder, baselines B0-B2, bias correction	Baseline errors table
6 - 9	LightGBM models, conformal, reconcile, first <code>/forecast/*</code> endpoints	Panchayat forecast with bands
10 - 11	Agro-variables, snapshots, risk and priority	Priority list
12 - 14	Advisory engine, templates, DB, review endpoints	Approve flow end to end
15 - 17	Verification, impact, explain	Reliability plot and B0 vs model table
18 - 19	Feedback, changes, audio, data-quality, seeds	Farmer journey
20	Tests, docs, snapshots for demo, Docker	Rehearsal ready

14.4 Switching from mock to real data

1. Replace `stations.csv` and `station_observations_*.csv` with real station data using the same columns. Run QC.
2. Replace `block_forecast_mock_nwp.csv` with real NWP aggregated to blocks (or official block forecasts if you truly have them). Same columns.
3. Replace static and satellite files with real per-Panchayat features and boundaries.
4. Set `DATA_MODE=real`. `load_truth()` now raises. Training targets come only from the station Panchayats.
5. Train with the same code, evaluate with leave-one-station-out (and leave-one-block-out where stations allow). With few stations, keep the model simple and say so.
6. Keep `data_mode` accurate in every response so the UI ribbon disappears only when data are truly real.

If real station coverage is too thin to train Panchayat-level models, a legitimate route is: train on a fine gridded reference (IMERG, ERA5-Land) coarsened to block scale, then validate only on real stations. Label it as a proxy method everywhere.

Appendix A. API contract v0.1 (shared, frozen Day 2)

This is the handshake between the two of you. The backend implements it. The frontend builds against it. Both sides write their tests from the same examples. Base URL in development: `http://localhost:8000/api/v1`.

A1. Conventions and enums

Item	Rule
IDs	panchayat_id like MP0103, block_id like MB01, station_id like MOCK_AWS_01, advisory id is a readable string
Variables (var)	rain (mm/day), tmax (C), tmin (C), rh (% daily mean), wind (km/h)
Quantile object	{p10, p50, p90, block} where block is the block-level value for the same day. Rain p10 can be 0.
Risk type	heavy_rain, heat, frost, waterlogging, dry_spell
Level	low, moderate, high, severe (UI colours: green, yellow, orange, red as in IMD warning colours)
Advisory category	sowing, irrigation, spray, fertilizer, harvest, heat_stress, frost, waterlogging, dry_spell, pest_disease, livestock
Status	draft, approved, edited, rejected
Confidence	high, medium, low
Language	en, hi, pa
Nulls	Missing numbers are null, never NaN, never the string "NaN"
Errors	HTTP 4xx/5xx with body {"error": {"code": "not_found", "message": "Panchayat MP9999 does not exist"}}
Versioning	GET /meta returns api_version. Contract changes bump the minor version.
Auth	None in the prototype. Header X-Role: officer or farmer is sent by the UI for logging only. State this honestly in the demo.

A2. Endpoints

Method + path	Purpose	Used by
GET /health	Liveness check	both
GET /meta	District name, data_mode, api_version, available_issue_dates, languages, crops	all screens
GET /geo/panchayats	GeoJSON polygons (properties: panchayat_id, name, block_id)	map
GET /geo/blocks	GeoJSON block polygons	map
GET /forecast/map?issue_date&lead_day&var	Values for both map modes: block layer and Panchayat layer	map
GET /forecast/panchayat/{id}?issue_date	5-day detail with quantiles, event probabilities, derived agro-variables	detail panel, farmer app
GET /observed/panchayat/{id}?from&to	What actually happened (station or synthetic truth, with source)	forecast vs observed overlay
GET /explain/{id}?issue_date&lead_day&var	Top 3 plain-language reasons this Panchayat differs from its block	detail panel
GET /risk?issue_date&lead_day&type	Risk level and score per Panchayat for one risk type	risk map
GET /priority?issue_date&horizon_days	Ranked list of Panchayats needing attention	priority screen
GET /advisories?status&panchayat_id&issue_date	Advisory list for the review queue	review screen
GET /advisories/{id}	One advisory with evidence and audit trail	review screen
POST /advisories/{id}/review	Approve, edit or reject (writes audit log)	review screen
GET /audio/{advisory_id}?lang	MP3 of the advisory text (404 if not available)	farmer app
GET /farmers/{id}	Demo farmer profile: Panchayat, crops, sowing dates, language	farmer app
GET /farmers/{id}/advice?issue_date	Approved advisories relevant to this farmer	farmer app
POST /feedback	Farmer taps "Did it rain?"	farmer app
GET /forecast/changes/{id}?issue_date	What changed since the previous forecast and whether advice changed	farmer app, detail panel
GET /verification/summary	Model vs baselines B0, B1, B2 per variable, with confidence intervals	verification screen
GET /verification/reliability?event	Reliability diagram points for one event	verification screen
GET /verification/coverage	Nominal vs empirical interval coverage	verification screen
GET /verification/regions	Error by held-out block or station	verification screen
GET /impact?season	Replay of past advice: model vs block-level decisions	impact screen
GET /data-quality	Stations reporting, missing share, stale inputs	officer console

A3. Example payloads

Numbers below are structural examples only. The backend generates real ones from the mock data in `contract/examples/`.

GET /meta

```
{ "api_version": "0.1", "data_mode": "mock", "district": "Synthetic District",
  "available_issue_dates": ["2024-08-12", "2024-08-13", "2024-12-16"],
  "languages": ["en", "hi", "pa"], "crops": ["wheat", "paddy", "cotton", "mustard", "bajra"],
  "vars": [ {"var": "rain", "unit": "mm"}, {"var": "tmax", "unit": "C"}, {"var": "tmin", "unit": "C"},
    {"var": "rh", "unit": "%"}, {"var": "wind", "unit": "km/h"} ] }
```

GET /forecast/map?issue_date=2024-08-12&lead_day=1&var=rain

```
{ "issue_date": "2024-08-12", "valid_date": "2024-08-13", "lead_day": 1, "var": "rain", "unit": "mm",
  "block_layer": [ {"block_id": "MB01", "value": 12.4} ],
  "panchayat_layer": [ {"panchayat_id": "MP0103", "block_id": "MB01", "p10": 0.0, "p50": 9.8, "p90":
    31.0,
    "block_value": 12.4, "delta": -2.6, "prob_event": 0.71, "event":
      "rain_ge_2_5mm"} ] }
```

GET /forecast/panchayat/MP0103?issue_date=2024-08-12

```
{ "panchayat_id": "MP0103", "name": "Synthetic Panchayat MP0103", "block_id": "MB01",
  "issue_date": "2024-08-12", "data_mode": "mock",
  "static": { "elevation_m": 214.8, "soil_texture": "clay_loam", "drainage_class": "poor",
    "irrigated_frac": 0.62 },
  "days": [ {
    "date": "2024-08-13", "lead_day": 1,
    "rain": { "p10": 0.0, "p50": 9.8, "p90": 31.0, "block": 12.4},
    "tmax": { "p10": 32.1, "p50": 33.4, "p90": 35.0, "block": 33.9},
    "tmin": { "p10": 24.0, "p50": 25.1, "p90": 26.2, "block": 25.3},
    "rh": { "p10": 66, "p50": 74, "p90": 82, "block": 72},
    "wind": { "p10": 4.1, "p50": 7.3, "p90": 11.0, "block": 7.9},
    "prob": { "rain_ge_1mm": 0.78, "rain_ge_2_5mm": 0.71, "rain_ge_10mm": 0.42, "rain_ge_35mm": 0.06},
    "derived": { "et0_mm": 4.6, "soil_moisture_frac": 0.81, "thi": 78.2, "waterlog_risk": "moderate"} }
  ] }
```

GET /advisories/{id}

```
{ "id": "ADV-2024-12-16-MP0103-wheat-irrigation", "issue_date": "2024-12-16",
  "panchayat_id": "MP0103", "block_id": "MB01", "crop": "wheat", "stage": "jointing_booting",
  "category": "irrigation", "priority": "high", "valid_from": "2024-12-17", "valid_to": "2024-12-19",
  "action": { "en": "Irrigate tomorrow morning. Skip if you already irrigated in the last 3 days.",
    "hi": "...", "pa": "..."},
  "reason": { "en": "Soil is drying and rain is unlikely before Friday.", "hi": "...", "pa": "..."},
  "fallback": { "en": "If rain above 5 mm arrives overnight, skip irrigation.", "hi": "...", "pa":
    "..."},
  "confidence": "medium",
  "evidence": [ { "label": "Chance of rain over 5 mm, next 3 days", "value": "18%"},
    { "label": "Soil water left in root zone", "value": "41%"} ],
  "thresholds_status": "placeholder",
  "status": "draft", "reviewed_by": null, "reviewed_at": null,
  "audit": [ {"at": "2024-12-16T08:02:11", "actor": "system", "action": "created", "note": ""} ],
  "audio": { "hi": "/api/v1/audio/ADV-2024-12-16-MP0103-wheat-irrigation?lang=hi" } }
```

POST /advisories/{id}/review (request) and POST /feedback (request)

```
{ "action": "edit", "reviewer": "Officer Demo", "note": "Shortened wording",
  "edited": { "action": { "en": "...", "hi": "...", "pa": "..."} } } <- action: approve | edit
| reject

{ "panchayat_id": "MP0103", "date": "2024-08-13", "reported_rain": true, "intensity": "moderate",
  "channel": "app" } <- intensity: none | light
| moderate | heavy
```

GET /priority, /explain, /verification/summary, /verification/reliability, /impact

```
/priority -> { "issue_date": "...", "items": [ { "panchayat_id": "MP0103", "block_id": "MB01",  
  "top_risk": "waterlogging", "level": "high", "score": 0.74, "crops_affected": ["paddy"],  
  "headline": {"en": "Heavy rain likely tomorrow on poorly drained soil"} } ] }  
  
/explain -> { "panchayat_id": "MP0103", "var": "tmax", "delta_vs_block": -0.5,  
  "reasons": [ {"feature": "irrigated_frac", "effect": "cooler", "text": {"en": "More irrigated fields  
    around it"}} ] }  
  
/verification/summary -> { "data_mode": "mock", "method": "leave_one_block_out + temporal_holdout",  
  "period": {"start": "2024-07-16", "end": "2024-12-31"},  
  "variables": [ { "var": "tmax", "n": <int>, "metrics": [ { "name": "MAE", "unit": "C",  
    "model": <num>, "b0": <num>, "b1": <num>, "b2": <num>, "skill_vs_b0": <num>,  
    "skill_ci95": [<num>, <num>] } ] } ],  
  "events": [ { "event": "rain_ge_2_5mm", "pod": <num>, "far": <num>, "csi": <num>, "brier": <num>,  
    "brier_skill_vs_climatology": <num> } ],  
  "block_mean_error": {"rain": <num>, "tmax": <num>}, "notes": ["Proxy validation on synthetic truth"]  
  }  
  
/verification/reliability?event=rain_ge_2_5mm -> { "points": [ {"forecast_prob": 0.1, "observed_freq":  
  0.08, "n": 812} ] }  
  
/impact?season=monsoon_2024 -> { "decision": "spray", "model": {"correct": <int>, "wasted_wait": <int>,  
  "washed_off": <int>},  
  "block_baseline": {"correct": <int>, "wasted_wait": <int>, "washed_off": <int>}, "n_decisions":  
  <int> }
```

Placeholders written as <num> or <int> are filled by the verification job. Nobody types accuracy numbers by hand.

Appendix B. How the two halves are integrated

Integration is not a final-week event. You integrate in small steps, one endpoint at a time, from Week 1. The idea: the frontend never waits for the backend, and the backend never guesses what the frontend needs.

B1. The five integration mechanisms

1. **Shared contract folder.** `contract/openapi.json` (generated by FastAPI) and `contract/examples/*.json` (hand-checked). Both people read from it.
2. **Example JSON as the first backend.** On Day 2 the backend person exports one example file per endpoint using mock data (even crude). The frontend serves them as a fake API and builds every screen from them.
3. **Generated types.** Frontend runs `npm run gen:types` (uses `openapi-typescript` on `contract/openapi.json`). If the backend changes a field, the frontend build fails immediately instead of showing a blank chart in the demo.
4. **Endpoint-by-endpoint switch.** The frontend has one env variable listing which endpoints use real data: `VITE_REAL_ENDPOINTS=meta,geo,forecast`. Everything else still reads examples. This makes progress visible and bugs local.
5. **Contract tests on both sides.** Backend tests check each real response validates against its Pydantic model and equals the example's shape. Frontend tests parse each example with its type guards.

B2. Milestones

Milestone	Backend delivers	Frontend delivers	Joint check
M0 Day 1-2	Repo, data loading, contract examples	Repo, design tokens, app shell, mock API	Both run locally. Contract frozen.
M1 end Week 1	<code>/meta</code> , <code>/geo/*</code> , <code>/forecast/map</code> (baseline B0 values are fine)	Map screen with block/Panchayat toggle	Map shows real polygons from the API
M2 mid Week 2	Model forecasts, quantiles, <code>/forecast/panchayat</code> , <code>/observed</code>	Detail panel, fan chart, observed overlay	Forecast vs observed for one Panchayat looks sensible
M3 end Week 2	Advisory engine, DB, review endpoints, <code>/priority</code> , <code>/risk</code>	Review queue, priority list, risk map, advisory card	Officer approves a card that then appears on farmer side
M4 mid Week 3	Verification, impact, explain endpoints	Verification and impact screens	Reliability plot and B0 vs model table are real
M5 end Week 3	Feedback, changes, audio, data-quality	Farmer app, language toggle, audio, bulletin print page	Full farmer journey works in three languages
M6 Week 4	Hardening, docs, snapshots	Polish, accessibility, offline snapshot	Two full dry runs of the demo script

B3. Running both together

```
# terminal 1 - backend
cd backend && source .venv/bin/activate
uvicorn gramdrishti.api.main:app --reload --port 8000

# terminal 2 - frontend
cd frontend && npm run dev # http://localhost:5173

# frontend/.env.local
VITE_API_BASE=http://localhost:8000/api/v1
VITE_REAL_ENDPOINTS=meta,geo,forecast # grow this list as endpoints go live

# export the contract after any backend API change (backend person)
python -m gramdrishti.export_openapi # writes contract/openapi.json
# regenerate frontend types (frontend person)
npm run gen:types # openapi-typescript ../contract/openapi.json -o
src/api/schema.d.ts
```

- **CORS:** the backend allows `http://localhost:5173` (and the deployed frontend origin later). If the browser shows a red CORS error, this is almost always why.
- **Docker (Week 4):** one `docker-compose.yml` with two services, `api` (port 8000) and `web` (nginx serving the built frontend, proxying `/api` to `api`). This gives you a single command for the demo laptop.
- **Demo laptop rule:** the final demo runs from the same machine setup you rehearsed on. Do not update packages the night before.

B4. Integration checklist (tick each before the demo)

Check	Owner	Done
Every example JSON validates against its Pydantic model	Backend	
<code>openapi.json</code> regenerated after last API change; types regenerated	Both	
No NaN or Infinity anywhere in real responses (search with curl)	Backend	
GeoJSON is <code>[longitude, latitude]</code> ; map renders in the right place	Both	
Dates and time zones: all ISO dates, no time-zone shifts on the map slider	Frontend	
Panchayat quantile order <code>p10 <= p50 <= p90</code> everywhere	Backend	
Block-mean of Panchayat values equals block value (consistency test passes)	Backend	
Mock ribbon visible on every screen when <code>data_mode = mock</code>	Frontend	
Empty, loading and error states exist for each screen	Frontend	
Approve on officer screen changes what the farmer app shows	Both	
Farmer feedback tap is stored and visible in database	Both	
All three languages render (fonts loaded, no boxes) on the demo laptop	Frontend	
Audio plays, or fallback to browser speech works offline	Both	
Offline snapshot mode works if the API is killed	Frontend	
Verification numbers on screen equal <code>artifacts/verification.json</code>	Both	

B5. Common integration bugs and fixes

Symptom	Likely cause	Fix
Map polygons appear in the ocean or flipped	Coordinates written <code>[lat, lon]</code>	GeoJSON must be <code>[lon, lat]</code> ; fix in backend export
Chart shows blank line	<code>null</code> values not handled	Frontend filters nulls; backend uses <code>null</code> not <code>NaN</code>
Colors all the same on map	Feature-state values not joined by id	Use <code>promoteId: 'panchayat_id'</code> and matching ids
Panchayat forecast differs from map value	Two different code paths in backend	Both endpoints read the same snapshot table
Types compile locally but fail on other laptop	Stale <code>schema.d.ts</code>	Regenerate types after <code>git pull</code>
API slow (seconds per click)	Model inference on each request	Precompute per issue date into snapshots; API only reads
Hindi or Punjabi shows boxes	Font not loaded or not subset for script	Load Noto Sans Devanagari and Gurmukhi; test offline copy
Advisory approved but farmer app unchanged	Farmer endpoint returns drafts too, or cache	Filter status in <code>(approved, edited)</code> ; invalidate query cache after review

B6. Demo insurance and roles

- **Snapshot mode.** Backend exports contract/snapshot/ (all JSON for the demo issue dates). Frontend can run with `VITE_SNAPSHOT=1` and needs no server at all. If the live API breaks on stage, switch and continue.
- **Two dry runs** at least, timed, with the jury questions list from docs/qna.md (write it together).
- **Roles on stage:** the frontend person drives the screens. The backend person explains the model, the uncertainty and the validation. Both can answer the other's questions in one sentence.
- **Freeze:** no feature work in the last 48 hours. Only fixes.

B7. Demo script (about 6 minutes)

1. Open the map on a monsoon issue date. Show the block forecast as one flat colour. Toggle to Panchayat view. Show the spread and the uncertainty band.
2. Click two Panchayats in the same block. Show different forecasts, then different advisories for the same crop, each with a reason.
3. Open the observed overlay to show what really happened (mock: synthetic truth, say so).
4. Switch to the Review screen. Edit one advisory, approve it. Show the audit trail.
5. Switch to the farmer phone view. Show the approved card in Hindi, play the audio, tap "Did it rain?".
6. Open Verification: table against B0, B1, B2 with confidence intervals, reliability plot, and the honest note about where the model does not help.
7. Open Impact: replay of the spray and heat decisions, model versus block level.
8. Close with the mock-to-real plan: same contract, real files swapped in.